

Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA
STEI ITB

IF2224 - Formal Language and Automata Theory

Arion Compiler

Milestone 3 - Semantic Analysis

Milestone 3

Semantic Analysis

Release : Kamis, 7 Mei 2026

Deadline : Kamis, 21 Mei 2025 pukul 23.59 WIB

Revisi : -

Daftar Revisi

[DD-MM-YY] Revisi X. Isi Revisi

Daftar Isi

Daftar Revisi.....	2
Daftar Isi.....	4
I. Teori Singkat.....	5
A. Abstract Syntax Tree (AST).....	6
B. Decorated Abstract Syntax Tree (Decorated AST).....	7
II. Spesifikasi Tugas Besar.....	9
A. Spesifikasi Program.....	9
B. Predefined Identifier.....	14
C. Type Compatibility.....	14
D. Contoh Input/Output.....	17
III. Teknis Pengerjaan.....	25
IV. Pengumpulan.....	27
V. Penilaian.....	28
Referensi.....	30
Lampiran A. Link Penting.....	31
Lampiran B. Contoh Symbol Table.....	32
Lampiran C. Summary of Operators.....	35
Lampiran D. Reserved Words.....	37

I. Teori Singkat

Pada milestone I lexical analysis, Anda berhasil mengubah source code menjadi bentuk token-token kecil dengan aturan DFA. Pada milestone II parser, dalam proses pengecekan syntax dengan Recursive Descent, Anda berhasil mengubah token-token tersebut menjadi sebuah parse tree.

Namun perlu diperhatikan bahwa sekedar pengecekan syntax tidak cukup untuk memastikan kebenaran dari program. Oleh karena itu, kita perlu **melakukan pengecekan makna (semantic)** dari program, yaitu pada tahapan Semantic Analysis.

Pengecekan semantic yang dilakukan oleh **Semantic Analysis** terdiri dari:

- **Type Checking (Pengecekan Tipe)**

Memastikan operator dan operand memiliki tipe yang kompatibel.

contoh: tidak menjumlahkan string dengan integer, perbandingan char terhadap string.

- **Symbol Table Management (Pengecekan Deklarasi)**

Memastikan setiap identifier (variabel/fungsi) telah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam cakupan yang sama.

- **Scope Resolution (Pengecekan Lingkup)**

Menentukan validitas akses variabel berdasarkan hierarki blok kode.

Contoh: scope global vs scope local.

- **Control Flow Validation (Pengecekan Alur Kontrol)**

Memastikan struktur alur kontrol yang logis.

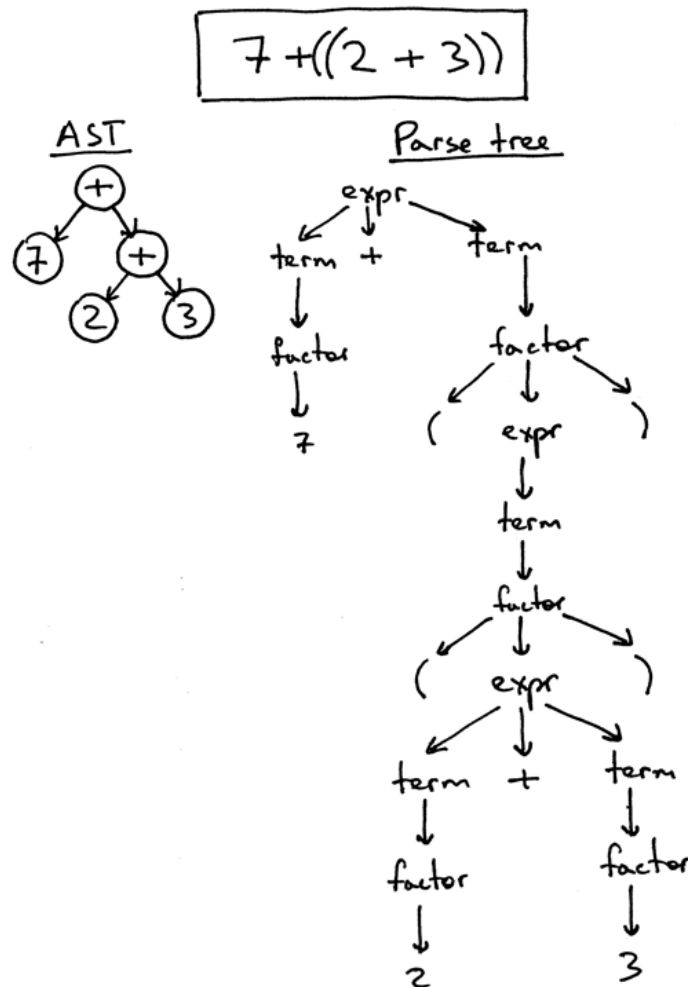
Contoh: instruksi break/continue hanya dalam loop, fungsi dipastikan memiliki return pada semua cabang kontrol, warning untuk unreachable code.

Semantic Analysis akan melakukan anotasi terhadap parse tree yang telah dihasilkan oleh parser dan menghasilkan sebuah **decorated abstract syntax tree (AST)**. Anotasi berupa (dan tak terbatas kepada) penambahan informasi, seperti tipe data dan referensi simbol (identifier) ke symbol table.

A. Abstract Syntax Tree (AST)

Abstract Syntax Tree pada dasarnya adalah **bentuk ringkas dari parse tree**, dimana kita hanya akan menyimpan informasi yang diperlukan untuk tahapan selanjutnya dan membuang informasi yang sudah tidak lagi diperlukan.

Struktur dasar sebuah **parse tree**, didasarkan pada **aturan Grammar** untuk melakukan pengecekan syntax. Namun pada tahap **Semantic Analisis** dan selanjutnya, kita tidak lagi memerlukan konteks tersebut. Sehingga pada **AST**, kita berpindah fokus menuju **aturan semantic**, yaitu mengenai **hubungan antar operator dan operand**. Aturan tersebut akan menjadi penting untuk pengecekan semantic dan tahapan selanjutnya.

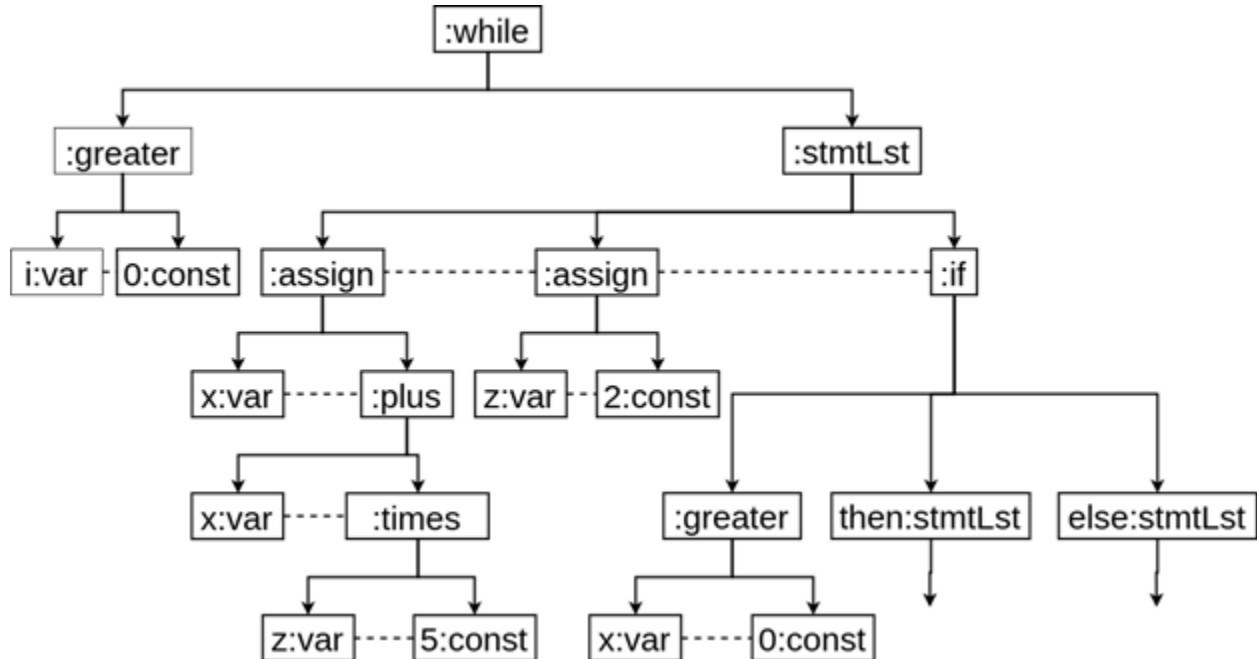


Gambar Parse Tree dan Abstract Syntax Tree (AST) untuk ekspresi matematika
(Sumber: <https://ruslanspivak.com/lbasi-part7>)

Sebagai ilustrasi, untuk potongan kode berikut.

```
/* TIDAK DIPASTIKAN SESUAI BAHASA ARION */
while (i > 0) {
    x = x + z * 5;
    z = 2;
    if (x > 0) then { ... } else { ... }
}
```

Kode tersebut akan menghasilkan Abstract Syntax Tree:



Gambar Contoh Abstract Syntax Tree (AST)
(Sumber: <https://nus-cs3203.github.io>)

B. Decorated Abstract Syntax Tree (Decorated AST)

Semantic Analysis akan melakukan **anotasi** terhadap AST hasil konversi dari parse tree untuk menghasilkan sebuah **Decorated AST**. Anotasi yang dilakukan bertujuan memberikan konteks untuk **pengecekan semantic**, seperti penambahan simbol table, tipe data, penambahan reserve keywords, pelacakan identifier, penanda tingkatan scope, dan sebagainya. Implementasi Decorated AST memang beragam, namun dalam tugas ini, kita akan berbasis **3 symbol table**, yaitu **tab**, **btab**, dan **atab** yang dijelaskan dalam [spesifikasi tugas](#). Jadi Decorated AST merupakan hasil dari proses pengecekan semantic pada Semantic Analysis.

Berikut adalah beberapa istilah yang perlu dipahami:

1. Attributed Grammar

Grammar yang dilengkapi dengan atribut dan aturan semantik.

2. Symbol Table

Struktur data yang minimal menyimpan informasi nama identifier, tipe datanya dan scopenya. **Symbol table menggunakan struktur data stack.**

3. Visitor / Visit Functions

Untuk setiap jenis node akan memiliki fungsi visit_<node> yang akan:

- 1) Memanggil visit pada child-nya
- 2) Mengakses atau memperbarui symbol table
- 3) Menghitung atribut/tipe semantik
- 4) Menambahkan anotasi pada node

II. Spesifikasi Tugas Besar

A. Spesifikasi Program

Pada milestone ini, Anda diminta untuk membuat **tahapan ketiga dari compiler**, yaitu **semantic analysis**. Tahapan ini berfungsi melakukan analisis makna (semantic analysis) dengan menggunakan **Attributed Grammar**.

Parse tree yang sudah dihasilkan akan ditelusuri secara top-down menggunakan **fungsi visit (semantic visitor)** pada setiap node-nya yang dibuat berdasarkan grammar yang sebelumnya didefinisikan. Selama proses ini, symbol table digunakan untuk menyimpan dan mengambil informasi deklarasi simbol/identifier.

Masukan	Parse Tree
Keluaran	Decorated AST, Symbol Table
Algoritma	L-Attributed Grammar

Pada Semantic Analysis, akan dilakukan beberapa tahapan berikut:

1. Konversi Parse Tree menjadi AST dengan Syntax-Directed Translation:

- a) Memanfaatkan Syntax-Directed Translation Scheme untuk menyisipkan semantic action/rule pada aturan produksi grammar. Semantic Action berfungsi untuk mengevaluasi nilai atribut dari masing-masing node. **Tuliskan semantic action yang Anda gunakan pada laporan.**
- b) Menjalankan action pada setiap node parse tree untuk membangun struktur AST yang ringkas dengan cara:
 - i) Membentuk node baru untuk AST (seperti AssignNode, BinOpNode) yang merepresentasikan hubungan semantik antar operator dan operand.

- ii) Menghilangkan informasi sintaksis yang sudah tidak diperlukan untuk tahap analisis (seperti tanda kurung berlebih, keyword terminal BEGIN, END, dan SEMICOLON).
- iii) Menggabungkan potongan node dari bawah ke atas dengan node level bawah akan diserahkan ke node induknya untuk dirakit menjadi satu kesatuan AST yang utuh.

2. Proses Dekorasi AST (Type & Scope Checking)

Penelusuran secara Depth-First Search (DFS) menggunakan fungsi visit:

- a) Mengelola scope dengan melakukan Push pada stack Symbol Table (tabel tab dan btab) saat memasuki blok baru (seperti fungsi, prosedur, atau record) dan melakukan Pop saat keluar dari blok tersebut.
- b) Melakukan registrasi simbol dengan memasukkan detail identifier seperti nama, jenis objek, tipe data, level leksikal, dan alamat memori ke dalam tabel tab setiap kali ditemukan deklarasi.
- c) Melakukan lookup pada Symbol Table dari tingkat scope terdalam hingga terluar setiap kali menemukan penggunaan identifier untuk memastikan bahwa variabel atau fungsi tersebut sudah dideklarasikan sebelumnya.
- d) Melakukan inferensi dan validasi tipe data dengan langkah-langkah:
 - i) Mengkalkulasi tipe data hasil dari setiap operasi pada node (misalnya, operasi dua operand Integer akan menghasilkan tipe Integer).
 - ii) Memastikan kompatibilitas tipe data (berdasarkan aturan [Type Compatibility](#)) antara variabel tujuan dengan nilai ekspresi yang diberikan.
- e) Anotasi setiap node AST dengan informasi tambahan:
 - i) Tipe data akhir dari ekspresi tersebut.
 - ii) Referensi atau indeks yang mengarah pada entri yang tepat di Symbol Table.
 - iii) Tingkatan lexical level atau scope tempat node tersebut berada.

Sebelum proses dimulai, lakukan inisiasi symbol table yang terdiri dari 3 jenis:

1. **tab**: digunakan untuk menyimpan informasi mengenai identifier termasuk konstanta, variabel, prosedur, atau fungsi.
2. **btab**: digunakan untuk menyimpan informasi blok prosedur dan definisi tipe record.
3. **atab**: digunakan untuk menyimpan informasi array seperti tipe index, batasan index, tipe elemen, dan ukuran array.

Contoh symbol table tersedia pada [Lampiran B. Contoh Symbol Table](#)

Berikut merupakan daftar atribut dari symbol table

tab	
atribut	deskripsi
identifiers	Nama identifier (misalnya nama variabel, konstanta, tipe, prosedur, fungsi). Indeks dimulai dari 33 karena ada reserved words termasuk predefined identifiers.
link	Pointer/indeks ke identifier sebelumnya dalam scope yang sama. Digunakan untuk manajemen scope (linked list per blok).
obj	Kelas objek yang dienumerasi: konstanta, variabel, tipe, prosedur, fungsi, dll.
type	Tipe dasar dari identifier, misalnya: integer, boolean, char, real, array, record, dll. Biasanya berupa kode numerik.
ref	Pointer/indeks ke tabel lain jika tipe adalah komposit (array/record). Mengarah ke atab (array table) atau btab (record/procedure block).
nrm	Menandai apakah identifier adalah variabel normal (=1) atau parameter by-reference (var parameter) (=0).
lev	Tingkat <i>lexical level</i> tempat identifier dideklarasikan (0 = global, 1 = dalam prosedur, 2 = dalam prosedur di dalam prosedur, dst).
adr	Makna tergantung jenis objek: offset variabel di stack frame, nilai konstanta, offset field record, alamat prosedur, atau ukuran/penanda lain.

atab	
atribut	deskripsi
arrays	Indeks entri array
xtyp	Tipe indeks array (misalnya integer). Berupa kode tipe dari tabel tab.
etyp	Tipe elemen array (misalnya integer). Berupa kode tipe dari tabel tab.
eref	Pointer/indeks ke detail tipe elemen jika elemen adalah tipe komposit (misalnya array dalam array, atau record). Mengarah ke

	atab atau btab.
low	Batas bawah indeks array (misalnya 1 pada array[1..10] atau 0 pada array[0..15]).
high	Batas atas indeks array.
elsz	Ukuran satu elemen array (dalam byte/unit memori).
size	Total ukuran array

btab	
atribut	deskripsi
blocks	Indeks entri block (setiap block mewakili prosedur, fungsi, atau record type definition).
last	Pointer/indeks ke identifier terakhir yang dideklarasikan di dalam block tersebut (menghubungkan field record, parameter, atau variabel lokal).
lpar	Pointer/indeks ke parameter terakhir dari prosedur/fungsi pada block tersebut. Jika block adalah record, nilainya 0.
psze	Total ukuran parameter block (dalam byte/unit memori).
vsze	Total ukuran variabel lokal block (dalam byte/unit memori)

Dengan memiliki informasi pada symbol table, dapat dipastikan konsistensi tipe data serta memastikan variabel tidak bisa dioperasikan jika belum diinisialisasi dengan nilai.

B. Predefined Identifier

Terdapat beberapa ident yang harus sudah terdefinisi di dalam symbol table, sebagai berikut:

Identifier	Keterangan
Real	Memiliki type Real e.g. someArray[1..10] of Real
Integer	Memiliki type Integer e.g. procedure print(x: integer);
Char	Memiliki type Char e.g. var a: Char
Boolean	Memiliki type Boolean e.g. var b: Boolean
String	Memiliki type String e.g. var c: String
True	Memiliki type Boolean dengan nilai <i>true</i> e.g. const isTrue == true
False	Memiliki type Boolean dengan nilai <i>false</i> e.g. if (someVar = false) then

Anda boleh menambahkan predefined identifier lain jika diperlukan (e.g. fungsi Readln, Writeln). **Tuliskan predefined Identifier yang Anda gunakan pada laporan.**

C. Type Compatibility

Suatu "type" menentukan "bentuk data" yang dapat ditampung constant (variable), expression, functions, dll. Type Compatibility Rule menentukan bagaimana type dapat digunakan di dalam expression, statements (e.g. assignments), dll.

Pada Milestone ini, Anda diminta untuk menentukan Type Compatibility Rule yang akan dipakai di bahasa Arion. Hal ini mencakup:

1. Menentukan aturan khusus mengenai type yang harus digunakan oleh suatu Node (e.g. expression di dalam if-statement harus bertipe Boolean);
2. Aturan interaksi antara dua type (e.g. constant di dalam range harus memiliki type yang sama).
3. Semantic Rule yang berhubungan dengan type (e.g. constant yang diproduksi dari realcon memiliki type Real/constant_real)

Terdapat beberapa type dan rule dasar yang harus Anda definisikan. Type dan rule dasar berikut dapat Anda tambahkan dan kembangkan lebih lanjut. **Tuliskan Type Compatibility Rule yang Anda gunakan di dalam Laporan.**

Type dasar, *simple type*, yang digunakan dalam bahasa pemrograman Arion adalah sebagai berikut:

No	Nama Type	Keterangan
	Real	realcon memiliki type Real
	Integer	intcon memiliki type Integer
	Char	charcon memiliki type Char
	Boolean	term dan expression yang diproduksi dari notsy , andsy , dan orsy memiliki type Boolean expression yang diproduksi dari relational-operator memiliki type Boolean
	String	string memiliki type String
	Subrange	Type dari Node Range ditentukan oleh type dari Constant di dalamnya. Lower Bound dari Range (Constant di kiri) tidak boleh lebih besar daripada Upper Bound dari Range. Subrange tidak boleh memiliki type Real.
	Enumerated	Type dari Node Enumerated ditentukan oleh type dari Ident di dalamnya. Ident di dalam enumerated harus memiliki type yang sama.

Terdapat juga *structured type* dengan contoh aturan sebagai berikut:

No	Nama Type	Keterangan
	Array	Array memiliki struktur berikut: array [T1] of T2 Dengan T1 adalah index type dan T2 adalah element type. Index type harus memiliki <i>simple type</i> bukan Real. Sedangkan T2 dapat berupa type apapun.
	Record	Field di dalam Record dapat memiliki type apapun
	Structured type dapat bersifat <i>named</i>, jika didefinisikan di dalam type-declaration, atau <i>anonymous</i>, jika didefinisikan secara langsung (e.g. di dalam var-declaration). Type dari <i>named</i> record ditentukan berdasarkan nama ident di dalam type-declaration. Dua <i>anonymous</i> record, meskipun memiliki struktur yang sama, bersifat <i>incompatible</i>. E.g.: <pre>var point1: record x, y: Integer; end; point2: record x, y: Integer; end; begin point1.x := 10; {Incompatible Error} point2 := point1; end;</pre>	

Adapun Type Compatibility Rule dasar sebagai berikut:

- a. Dua type adalah compatible jika memenuhi salah satu dari kondisi berikut:
 1. Kedua type memiliki type yang sama
 2. Satu type adalah subrange dari yang lain (e.g. [1..10] dengan Integer 5), atau kedua type adalah subrange dari type yang sama (e.g. [1..10] dengan [50..100]).
 3. Kedua type memiliki type String dan memiliki panjang yang sama
- b. Sebuah type T2 adalah *assignment-compatible* dengan type T1 jika memenuhi salah satu dari kondisi berikut:
 - i. T1 adalah Real dan T2 adalah Integer

- ii. T1 dan T2 adalah compatible, bertipe Integer, Boolean, atau Char, dan nilai (value) yang dimiliki T2 adalah subset dari nilai yang dimiliki T1 (e.g. T2 = 5 dan T1 = [1..10])
- iii. determined by T 1.
- iv. T1 and T2 adalah compatible dan bertipe String
- c. Suatu pasangan ident dan expression dapat digunakan dalam assignment-statement jika ident dan expression tersebut adalah *assignment-compatible*, sebagai contoh:

```

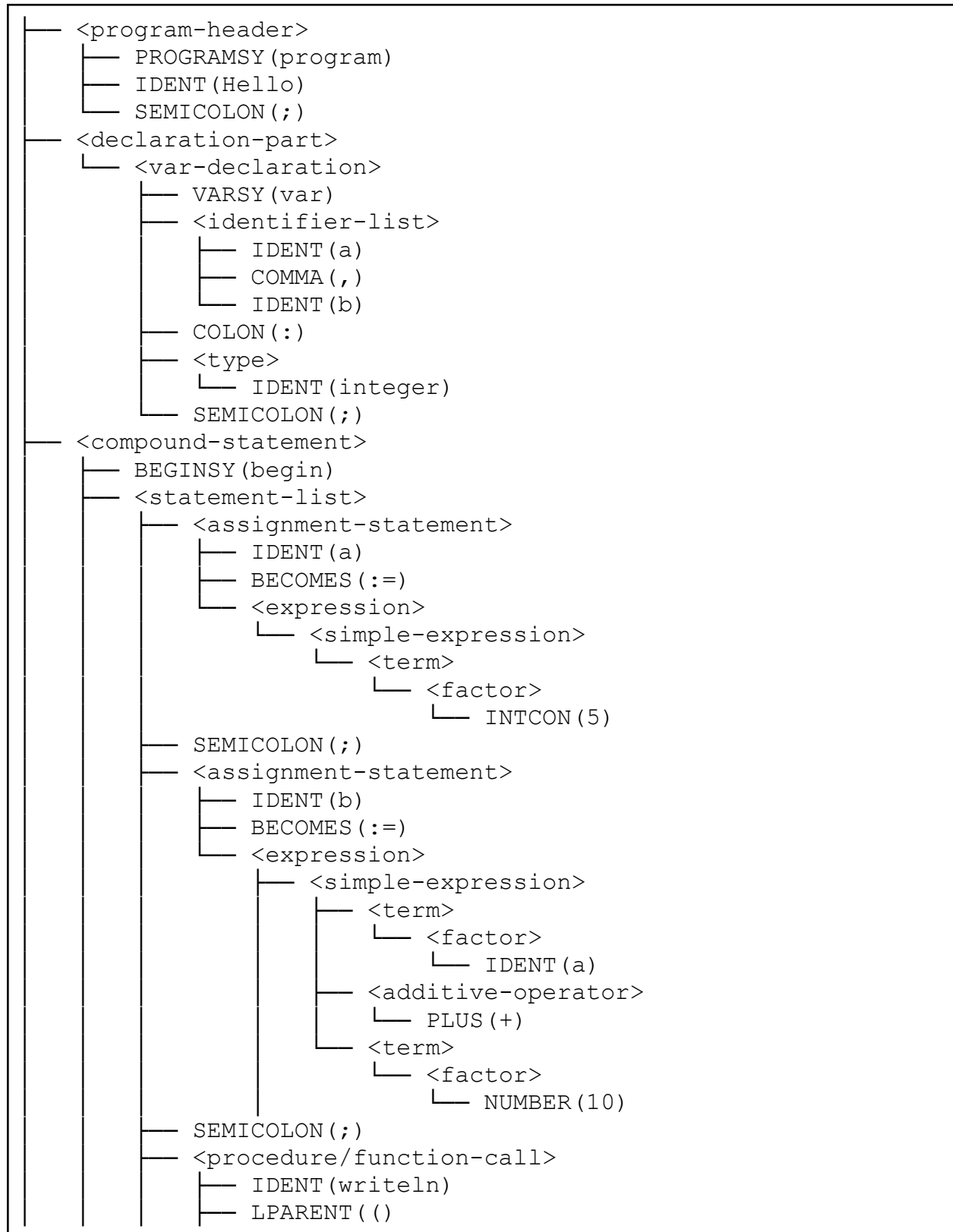
var
  index: 1..10;
  count: Integer;
begin
  count := 5;
  index := count;

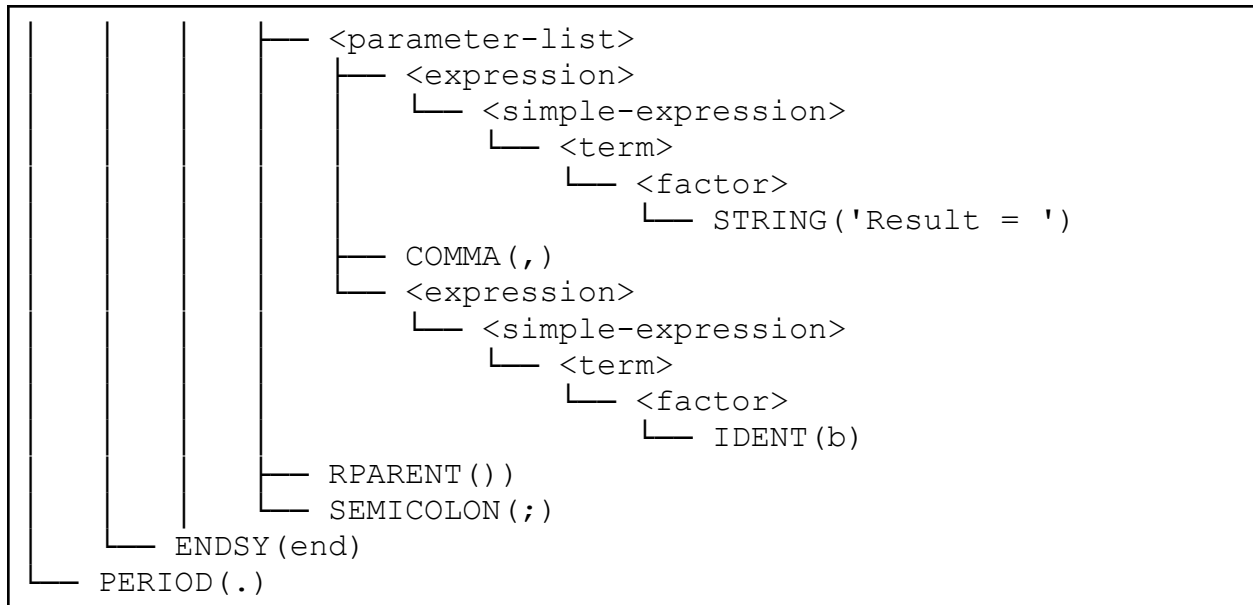
  { index := 11; } { Assignment-Incompatible Error }
end;

```

D. Contoh Input/Output

Isi program.txt
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end. </pre>
Masukan (Input) untuk fase semantic analysis
Parse tree yang dihasilkan oleh parser
Isi Parse tree yang sebelumnya dihasilkan oleh parser
<program>



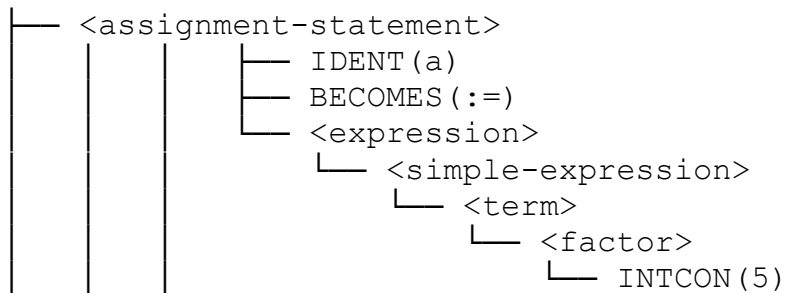


Contoh Semantic Rule yang digunakan (Silahkan ditentukan sendiri)

Production Rule	Semantic Rule
$\langle \text{assignment-statement} \rangle \rightarrow \text{ID} := \langle \text{expr} \rangle$	<code>assignment_statement.node = new AssignNode(new VarNode(ID.lexeme), expr.node)</code>
$\langle \text{expression} \rangle \rightarrow \langle \text{simple-expr} \rangle + \langle \text{term} \rangle$	<code>expression.node = new BinOpNode('+', simple_expr.node, term.node)</code>
$\langle \text{expression} \rangle \rightarrow \langle \text{simple-expr} \rangle$	<code>expression.node = simple_expr.node</code>
$\langle \text{factor} \rangle \rightarrow \text{NUMBER}$	<code>factor.node = new NumberNode(NUMBER.value)</code>
$\langle \text{factor} \rangle \rightarrow \text{ID}$	<code>factor.node = new VarNode(ID.lexeme)</code>
$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle$	<code>term.node = factor.node</code>

<code><factor> → STRING_LITERAL</code>	<code>factor.node = new StringNode(STRING_LITERAL.value)</code>
<code><procedure-call> → ID (<params>)</code>	<code>procedure_call.node = new ProcCallNode(ID.lexeme, params.nodes)</code>

Proses Pembangunan AST



Misalkan Syntax Analyzer sedang memeriksa sub-tree diatas dan masuk sampai kedalam node "INTCON(5)"

Production Rule yang digunakan adalah `<factor> → ID` dengan semantic rule `factor.node = new VarNode(ID.lexeme)`

Maka, aksi yang dilakukan adalah
`factor.node = new NumberNode(5)`
 dan terbentuk node baru pada AST yaitu
`NumberNode(5)`

Kemudian naik pada node "<term>"

Production rule yang digunakan `<term> → <factor>`

Maka digunakan `term.node = factor.node` yang berakibat
`term.node = NumberNode(5)`

Atribut dinaikkan dalam tree (synthetized) sampai diperoleh
 bahwa `expression.node = NumberNode(5)`

Sekarang Syntax Analyzer naik sampai node
`<assignment-statement>` dimana production rulanya

`<assignment-statement> → ID := <expr>`

Semantic rulanya adalah

`assignment_statement.node = new AssignNode(new
VarNode(ID.lexeme), expr.node)`

Karena node ini punya leaf IDENT(a) dan attribut synthesized

dari <expression> maka terjadi
`assignment_statement.node = new AssignNode(new VarNode('a'),
 NumberNode(5))`
 Dan terbentuk node AST baru yaitu
`AssignNode(target: VarNode('a'), value: NumberNode(5))`

AST yang dihasilkan

```
ProgramNode(name: 'Hello')
|
+-- Declarations
|   |
|   +-- VarDecl(name: 'a', type: 'integer')
|   \-- VarDecl(name: 'b', type: 'integer')
|
\-- Block
    |
    +-- Assign(target: Var('a'), value: Num(5))
    |
    +-- Assign(target: Var('b'),
    |           value: BinOp(op: '+',
    |                           left: Var('a'),
    |                           right: Num(10)))
    |
    \-- ProcedureCall(name: 'writeln',
    |                   args: [String('Result = '), Var('b')])
```

Proses Type & Scope Checking

Traversal dimulai dari root dengan fungsi `visit(ProgramNode)`.
 Sebelum memulai, inisialisasi ketiga tabel simbol:

```
tab = []    # identifier table (mulai dari indeks 33 untuk
reserved words)
btab = [    # block table - indeks 0 adalah program (global
block)
    { "last":0, "lpar":0, "psze":0, "vsze":0 }    # btab[0] =
global block
]
atab = []   # array table (kosong awal)

display = [0]    # display[level] = indeks btab dari block pada
level tersebut
level    = 0     # lexical level (0 = global)
```

Contoh proses untuk program Hello yang sama:

```

1. visit(ProgramNode name='Hello')
    • Buat block baru untuk program (global block) → sudah ada
    di btab[0]
    • Masukkan nama program ke tab:
      tab.append({
        "id": "Hello", "obj": "program", "type": 0, "ref": 0,
        "nrm": 1, "lev": 0, "adr": 0, "link": 0
      })
    • btab[0]["last"] = indeks terakhir di tab yang termasuk
    block ini

2. visit(Declarations)
    • visit(VarDecl 'a', type='integer')
      • Cari tipe integer di tab (reserved, misal indeks 1 =
      integer)
      • Masukkan ke tab:
        tab.append({
          "id": "a", "obj": "variable", "type": 1, "ref": 0,
          "nrm": 1, "lev": 0, "adr": 0, "link":
previous_var_in_block
        })
      • Update btab[0]["last"] = indeks baru ini
      • adr akan dihitung nanti saat code generation (ukuran
      tiap variabel = 1 untuk integer)

      • visit(VarDecl 'b', type='integer')
        → sama seperti a, link menunjuk ke entri a (linked-list
        dalam block)

3. visit(Block) - masuk ke compound statement (main block)
    • level += 1
    • Buat block baru di btab:
      btab.append({
        "last": 0, "lpar": 0, "psze": 0, "vsze": 0
      })
    • display.append(len(btab)-1) # simpan indeks block level
1

4. visit(Assign target='a', value=5)
    • visit(Var 'a')
      • Lookup identifier 'a':
        mulai dari block saat ini (display[level]=1), ikuti link
        di btab

```

```

    → tidak ada → turun ke level 0 → ketemu di tab[indeks_a]
    • Anotasi node: type = integer, tab_index = indeks_a, lev
= 0
    • visit(Number 5) → type = integer
    • Cek kompatibilitas tipe → ok
    • Tandai variabel a sebagai initialized (bisa ditambah field
atau flag terpisah)

5. visit(Assign target='b', value=BinOp('+', Var('a'), 10))
    • Sama seperti di atas untuk 'b'
    • BinOp '+': kedua operand integer → result type integer
    • Assignment: target type = value type → ok

6. visit(ProcedureCall 'writeln')
    • Lookup 'writeln' → predefined procedure (sudah ada di tab
indeks kecil)
    • Cek jumlah dan tipe parameter (string + integer) → sesuai

```

Keluaran (output)

tab (hanya sebagian yang relevan):

idx	id	obj	type	ref	nrm	lev	adr	link
... (reserved words 0-32)								
33	Hello	program	0	0	1	0	0	0
34	a	variable	1	0	1	0	0	0
35	b	variable	1	0	1	0	0	30
36	writeln	procedure	...	(predefined)				

btab:

idx	last	lpar	psze	vsze
0	35	0	0	2
1	0	0	0	0

← global block (program), vsze=2 (a dan b)

← main block (kosong variabel lokal)

atab: (kosong karena tidak ada array)

Decorated AST (contoh anotasi minimal):

ProgramNode(name: 'Hello')

├ Declarations

├ ┌ VarDecl('a')

→ tab_index:34, type:integer, lev:0

└ VarDecl('b')

→ tab_index:35, type:integer, lev:0

└ Block

→ block_index:1, lev:1

```
└─ Assign('a' := 5)    → type:void
    └─ target 'a'      → tab_index:34, type:integer
      └─ value 5        → type:integer
└─ Assign('b' := a+10) → type:void
    └─ target 'b'      → tab_index:35, type:integer
      └─ BinOp '+'      → type:integer
        └─ 'a'          → tab_index:34, type:integer
          └─ 10          → type:integer
└─ writeln(...)        → predefined, tab_index:36
```


III. Teknis Pengerjaan

- Anda **melanjutkan program hasil milestone 2** dan menambahkan komponen **semantic analysis** dengan Bahasa pemrograman yang **disamakan** dengan pengerjaan milestone sebelumnya (C/C++).
- Anda dipersilahkan untuk memperbaiki kesalahan dari milestone sebelumnya karena dapat mempengaruhi hasil dari semantic analysis. Namun tidak ada penilaian ulang untuk hasil milestone sebelumnya.
- Pengerjaan dilakukan pada **repositori yang sama** dengan **kelompok yang sama** sesuai [sheets pembagian kelompok](#).
- **Jangan lupa menginvite asisten masing-masing ke dalam repository agar tugas dapat dinilai.**
- **Laporan setidaknya berisi:**
 - Cover dengan foto kelompok menggantikan logo ITB.
 - Landasan Teori
 - Jelaskan teori-teori atau konsep yang berhubungan atau digunakan pada Milestone 3.
 - Perancangan & Implementasi
 - Implementasi yang ditambahkan dan berhubungan dengan Milestone 3.
 - Pengujian
 - Setidaknya 5 pengujian kasus unik.
 - Setiap pengujian harus menunjukkan hasil input/output program.
 - Bukti berupa screenshot input/output program.
 - Kesimpulan dan Saran
 - Lampiran berupa:
 - Link Release Repository Github.

- Pembagian Tugas (menunjukkan persentase kontribusi).
 - Referensi yang digunakan selama pengerjaan Milestone 3
- Jika ada pertanyaan lebih lanjut, silahkan ditanyakan pada [QNA](#).
- Link Pembagian Kelompok: [Sheets Pembagian Kelompok](#)

IV. Pengumpulan

- **Pengumpulan bersifat Hard Deadline**
- Pada setiap Milestone, Anda **WAJIB** membuat **Release**.
 - Gunakan format tag berupa **v0.X.Y** dengan **X** adalah **nomor milestone** dan **Y** adalah **nomor pengumpulan**.
 - Sebagai contoh, untuk milestone 1, release pertama, dan pengumpulan pertama adalah **v0.1.1**. Untuk pengumpulan selanjutnya karena revisi, gunakan tag **v0.1.2** dan seterusnya sesuai nomor pengumpulan. Untuk milestone selanjutnya, pengumpulan dimulai kembali dari satu yaitu **v0.2.1**, **v0.2.2**, dan seterusnya.
- **Jangan lupa menginvite asisten masing-masing ke dalam repository agar tugas dapat dinilai.**
- **Laporan** dikumpulkan dengan cara **meletakkan** hasil **PDF** dalam **folder doc** dengan format **Laporan-X-[KODE].pdf**, dimana **X** adalah **nomor milestone**. Pastikan isi laporan sudah sesuai dengan **teknis pengerjaan**.
- Link Pengumpulan: [Pengumpulan Milestone](#)
- Deadline milestone: **Kamis, 21 Mei 2026 pukul 23.59 WIB**

V. Penilaian

Program (70 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Implementasi visit functions	12	Semua visit function lengkap & sesuai grammar.
2.	Implementasi symbol table	10	Symbol table menyimpan atribut minimal & berfungsi dengan benar saat proses semantic analysis.
3.	Annotated AST	10	Parse tree sudah teranotasi sesuai aturan AST dan tipe datanya konsisten.
4.	Struktur/modularitas program	8	Kode ditulis dengan struktur modular menggunakan fungsi atau kelas yang terpisah berdasarkan tanggung jawab. Alur eksekusi program mudah diikuti dan tidak redundan
5.	Error handling	5	Program dapat menangani error dengan baik (pesan informatif dan error tidak menyebabkan crash)
6.	Penerimaan input	3	Program dapat membaca input parse tree hasil parser pada milestone 2
7.	Penghasilan output	2	Program mengeluarkan output Decorated/Annotated Parse Tree yang terformat dengan jelas ke terminal
8.	Dokumentasi/komentar	5	Kode diberi komentar yang informatif dan tidak berlebihan
9.	Pengujian	15	Program berhasil lolos seluruh kasus uji

Laporan (30 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Penjelasan konsep	20	Penjelasan mengenai konsep semantic analysis.
2.	Perancangan program	5	Penjelasan mengenai rancangan program yang mencakup teknologi yang digunakan, struktur program, fungsi/kelas utama, dan alur kerja program. Terdapat pula justifikasi atas pilihan implementasi
3.	Hasil implementasi	2	Dokumentasi mengenai hasil implementasi, dapat berupa tangkapan layar dengan penjelasan
4.	Test case	3	Dokumentasi mengenai pengujian mandiri (minimal 5 dan mencakup berbagai kasus)

Referensi

Gambar Parse Tree dan Abstract Syntax Tree (AST)

<https://ruslanspivak.com/lrbasi-part7>

Penjelasan AST

<https://nus-cs3203.github.io/course-website/contents/basic-spa-requirements/abstract-syntax-tree.html>

“Semantic Analysis (ULiège)”

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>

Lampiran A. Link Penting

Laman Pembagian Kelompok: [📁 Pembagian Kelompok - Tubes IF2224 TBFO](#)

Laman QnA Tugas Besar: [📁 QNA - Tubes IF2224 TBFO](#)

Form Pengumpulan: [Form Pengumpulan - Tubes IF2224 TBFO](#)

Lampiran B. Contoh Symbol Table

Contoh Symbol Table (Belum tentu sesuai dengan spesifikasi)

```

0  program test0(output);
0  const ten = 10; plus = '+';
0  type row = array [1..ten] of real;
0      complex = record re,im: real and :
0  var i,j: integer;
0      p: boolean;
0      z: complex;
0      matrix: array [-3..+3] of row;
0      pattern: array [1..5, 1..5] of char;
0

```

-16-

```

0  procedure dummy(var i: integer; var z: complex);
0      var u,v: row;
0      h1,h2: record c: complex; r: row
0      end ;
0
0      function null(x,y: real; z: complex): boolean;
0      var a: array ['a'..'z'] of complex;
0      u: char;
0      begin while x < y do x := x+1.0;
10      null := x=y
12      end (*null*) ;
15
15      begin p := null(h1.c,re, h2.c.im, z)
25      end (*dummy*) ;
27
27      begin i := 85; j := 51;
34      repeat
34          if i > j then i := i-j else j := j-i
46      until i = j;
53      writeln(i)
55      end .

```


tab yang dihasilkan

identifiers	link	obj	typ	ref	nrm	lev	adr
29 ten	0	0	1	0	0	1	10
30 plus	29	0	4	0	0	1	37
31 row	30	2	5	1	0	1	10
32 complex	31	2	6	3	0	1	2
33 re	0	1	2	0	1	2	0
34 im	33	1	2	0	1	2	1
35 i	32	1	1	0	1	1	5
36 j	35	1	1	0	1	1	6
37 p	36	1	3	0	1	1	7
38 z	37	1	6	3	1	1	8
39 matrix	38	1	5	2	1	1	10
40 pattern	39	1	5	3	1	1	80
41 dummy	40	3	0	4	1	1	16
42 i	0	1	1	0	0	2	5
43 z	42	1	6	3	0	2	6
44 u	43	1	5	1	1	2	7
45 v	44	1	5	1	1	2	17
46 h1	45	1	6	5	1	2	27
47 h2	46	1	6	5	1	2	39
48 c	0	1	6	3	1	3	0
49 r	48	1	5	1	1	3	2
50 null	47	4	3	6	1	2	0
51 x	0	1	2	0	1	3	5
52 y	51	1	2	0	1	3	6
53 z	52	1	6	3	1	3	7
54 a	53	1	5	5	1	3	9
55 u	54	1	4	0	1	3	61

btabs yang dihasilkan

blocks	last	lpar	psze	vsze
1	28	1	0	0
-17-				
2	41	28	5	105
3	34	0	0	2
4	50	43	7	51
5	49	0	0	12
6	55	53	9	62

atab yang dihasilkan

arrays	xtyp	etyp	eref	low	high	elsz	size
1	1	2	0	1	10	1	10
2	1	5	1	-3	3	10	70
3	1	5	4	1	5	5	25
4	1	4	0	1	5	1	5
5	4	6	3	1	26	2	52

Lampiran C. Summary of Operators

Arithmetic

<i>Operator</i>	<i>Operation</i>	<i>Type of Operands</i>	<i>Type of Result</i>
(unary) +	identity	Integer or Real	same as operand
(unary) −	sign inversion	Integer or Real	same as operand
+	addition	Integer or Real	Integer or Real
−	subtraction	Integer or Real	Integer or Real
*	multiplication	Integer or Real	Integer or Real
/	real division	Integer or Real	Real
div	integer division	Integer	Integer
mod	modulus	Integer	Integer

Relational

<i>Operator</i>	<i>Operation</i>	<i>Type of Operands</i>	<i>Type of Result</i>
=	equality	Integer, Real, Char, or String	Boolean
<>	inequality	Integer, Real, Char, or String	Boolean
<	less than	Integer, Real, Char, Char, or String	Boolean
<=	less than or equal	Integer, Real, Char, Char, or String	Boolean
>	greater than	Integer, Real, Char, Char, or String	Boolean
>=	greater than or equal	Integer, Real, Char, Char, or String	Boolean

Boolean

<i>Operator</i>	<i>Operation</i>	<i>Type of Operands</i>	<i>Type of Result</i>
not	negation	Boolean	Boolean
and	conjunction	Boolean	Boolean
or	disjunction	Boolean	Boolean

Lampiran D. Reserved Words

1. AND
2. ARRAY
3. BEGIN
4. CASE
5. CONST
6. DIV
7. DOWNTO
8. DO
9. ELSE
10. END
11. FOR
12. FUNCTION
13. IF
14. MOD
15. NOT
16. OF
17. OR
18. PROCEDURE
19. PROGRAM
20. RECORD
21. REPEAT
22. INTEGER
23. REAL
24. BOOLEAN
25. CHAR
26. STRING
27. THEN
28. TO
29. TYPE
30. UNTIL
31. VAR
32. WHILE